

Breakable Commitments: Unblocking Closed-Loop Replanning in Coupled Job-Shop Scheduling and Multi-Agent Path Finding

Anonymous submission

Abstract

Flexible job-shop scheduling (FJSP) and multi-agent path finding (MAPF) couple naturally in smart factories: machines process operations while AGVs move material over a shared grid. State-of-the-art stacks solve this coupled problem *once* and execute open-loop, and a growing line of work proposes closed-loop replanning—yet we find that in a full-stack implementation, **every** dynamic plan revision is silently unexecutable: the request encoder cannot express successors of in-flight commitments, so a single active transport makes a revision impossible to activate. We make four contributions. (1) A formal online specification (O-IFJSP-MAPF) of the coupled problem with *commitment-consistent revision* semantics that mirrors real execution machinery. (2) An empirical diagnosis on a coupled benchmark: one-shot CP-SAT plans are *passively robust* to outages and duration variance (4–14% degradation under a 200-step machine outage), which masks the broken revision pathway—until a job stream arrives, where throughput collapses by two orders of magnitude. (3) *Soft commitments*: a conservative *commitment projection* that maps each in-flight commitment (μ, l, r) to exact resource and source (μ, l) and a bounded release $\bar{r} \geq r$ governed by a single interpretable penalty parameter, with a two-level soundness guarantee—execution safety holds *unconditionally* under stack release semantics, plan-time validity under a bounded-delay assumption. The projection lifts revision activation from 53% to 99% across 2401 triggered revisions. (4) A 1209-episode factorial evaluation with paired significance tests that yields a provocative conclusion: **event-triggered revision loses to doing nothing in every disruption scenario we could construct**—one-shot plans absorb 200-step outages within 6% via slack, while conservative revision pays a 25–125% premium. Under streaming arrivals, soft commitments deliver reliability—zero revision failures versus 1–18 per cell, guaranteed completion versus starvation in the worst legacy cell, and 23–31% throughput gains on the largest shop—and a snapshot-based *value-aware trigger* converts the diagnosis into a working decision policy: replanning only 1.3% of the time, it improves on both never-replan and always-replan in this regime (2876 vs. 3018/3466 mean makespan, paired tests). Closed-loop

factory scheduling needs a when-to-replan decision policy, not a better trigger—and a simple one already suffices.

1 Introduction

Manufacturing execution increasingly couples two classical problems. On the shop floor, *flexible job-shop scheduling* assigns operations to eligible machines and sequences them. On the logistics floor, *multi-agent path finding* routes AGVs collision-free over a grid. The two layers interact through transport: the schedule emits a stream of pickup-and-delivery requests whose realized travel times depend on congestion the scheduler does not see; conversely, routing decisions determine when material becomes available for processing.

Practical systems therefore decompose the problem hierarchically, and the most recent integrated frameworks [9] plan once and repair only the routing layer at execution time—a *generate-then-refine* paradigm whose authors themselves list closing the loop as future work. Meanwhile, the replanning literature (plan repair, execution monitoring, robust execution) offers a rich vocabulary [5, 4] but rarely meets the coupled factory stack, and dynamic-scheduling studies that do model machine breakdowns abstract transport away to a fixed time matrix [10].

This paper asks a deliberately unglamorous question: *in a realistic coupled stack, what does it take to change the plan after execution has started?* The answer turns out to be a mechanism-level bottleneck rather than a policy-level one:

- We formalize the online coupled problem and its *commitment-consistent revision* semantics (§3), mirroring the ledger/activation machinery of a real engine (plan revisions, frozen operations, activation checks).
- We show empirically that the standard encoding of commitment consistency *categorically blocks* revision: any queued or in-transit predecessor renders the residual problem inexpressible, and activation is vetoed on 47% of trigger opportunities ($n=2401$).

revisions)—across disruption revision, job arrivals, and periodic refresh (§4).

- We introduce *soft commitments*: a bounded-release encoding with one interpretable penalty parameter, plus exact commitment-machine resolution from queue membership and transport destinations (§5). The encoding is client-side and opt-in (a ~ 90 -line patch in our implementation) and raises activation from 53% to 99%.
- We instantiate the (τ, κ) recovery-orchestrator policy space on top and evaluate on a 1209-episode benchmark (948-episode main factorial plus six sensitivity phases) spanning instance families, fleet sizes, disruption scenarios, and commitment modes (§7).

Our headline findings revise two intuitions. First, *one-shot plans are passively robust*: slack plus release-by-plan semantics absorb severe disruptions (a 200-step outage degrades makespan by only 4–14%), so the broken revision pathway is invisible until it is catastrophic—on the pre-repair stack, streaming arrivals starved entirely ($24.4 \rightarrow 0.24$ jobs/ksteps). Second, once revisions are unblocked and measured at scale (1209 episodes across seven factorial phases, paired tests at $p < 10^{-4}$), event-triggered revision *loses* to doing nothing in every disruption scenario: the value of closed-loop replanning in this stack is reliability under arrivals—zero revision failures, guaranteed completion, and 23–31% throughput gains on larger shops—not disruption response.

2 Related Work

Coupled scheduling and routing. Integrated FJSP with transport has a long OR tradition; recent surveys catalogue metaheuristic and exact approaches [17, 3] on classic instance families [2]. Learning-based dispatching transfers across instances [18, 16], and the closest coupled framework co-optimizes a JSP schedule with PBS routing under a one-shot paradigm [9]. These evaluations use single maps and open-loop execution; revision semantics are not studied.

MAPF and lifelong MAPF. Optimal and bounded-suboptimal search (CBS [14], EECBS [7]), scalable suboptimal search (LaCAM [12]), and learned policies (MAPF-GPT [1]) are benchmarked on standardized grid suites [15]; lifelong variants handle task streams with rolling horizons [11, 8] and repair operators [6]. Robust MAPF treats stochastic travel [13]. None of these model machine eligibility, processing durations, or plan-level commitments.

Replanning and plan repair. Plan stability, repair, and execution monitoring are classical concerns [5], with commitment-based satisficing planning studying what to fix [4]. Our contribution is orthogonal and practical: we

show where a production-grade coupled stack silently *forbids* repair—the commitment encoding itself—and give the minimal sound relaxation that unblocks it, turning the stability–responsiveness trade-off into a single measurable penalty parameter.

3 Online Coupled Problem and Revision Semantics

3.1 Static core (I-FJSP-MAPF)

Jobs $\mathcal{J}$ carry operation chains O_j ; each operation o has eligible machines $E(o)$ with durations $p_{o,m}$; machines occupy cells of a grid G ; a fleet $\mathcal{K}$ of unit-capacity AGVs moves one edge per step. A solution assigns (i) machines and sequences, (ii) transport tasks to AGVs, and (iii) collision-free spacetime trajectories π_k , minimizing makespan $C_{\max}$. The coupling constraints are the hand-offs: an operation may start only after its predecessor’s material *actually arrives*—a time determined by the routing layer.

3.2 Online setting (O-IFJSP-MAPF)

An exogenous disruption process ξ emits machine outages $\mathbf{m_down}(m, d)$, AGV outages, and temporary obstacles; a job stream releases jobs at times a_j . A closed-loop policy observes the state (remaining operations, machine/AGV status, in-flight tasks, current plan) and may, at any step, issue a *revision*.

Definition 1 (Feasible revision). A revision $\Pi_{t+1} = \rho(\Pi_t \mid s_t)$ is feasible iff it (i) agrees with the executed prefix; (ii) honors commitments —started operations stay on their machines, loaded AGVs deliver to their destinations; and (iii) is (approximately) optimal for the residual problem.

Definition 2 (Disruption regret). $\text{RRegret}(\pi; S) = C_{\max}(\pi, S) / C_{\max}(\pi, S_0)$ for scenario S against the empty scenario S_0 .

3.3 Commitment classes

At any instant each operation is in one of: COMPLETED, PROCESSING (machine busy), MACHINE_QUEUE (queued at a machine), TRANSPORT (material on an AGV), or PENDING (free to schedule). Only PENDING operations are reschedulable; the rest are frozen. The revision machinery must therefore encode, for the first reschedulable operation of each job, *when its input material exists* (release bound) and *where* (source cell). This seemingly clerical encoding is the paper’s protagonist.

4 Diagnosis: Commitments Block Revision

State-of-the-art coupled frameworks plan once and repair only the routing layer, listing closed-loop revision as future work [9]. We argue this is no accident but a consequence of how such stacks naturally honor commitments. The baseline—and, we contend, the standard—encoding of commitment consistency in a stack that revises through a residual problem is *encode-or-veto*: express exactly what is expressible, and veto the revision otherwise. It is a defensible design: it can never violate a commitment. We instrument a full-stack implementation of this baseline pattern and show what it costs. The residual-problem encoder works as follows for a job’s first reschedulable operation o with predecessor p :

- $p \in \text{PROCESSING}$: $\text{release} := \text{machine availability (handled)}$;
- $p \in \{\text{MACHINE_QUEUE}, \text{TRANSPORT}\}$: the encoder *cannot express* the release or source; it appends p (or o) to an **unresolved** list, and the artifact is marked **execution_ready=False** iff that list is nonempty;
- the solver then *refuses to activate* any such revision.

Since a running factory almost always has queued or in-transit work, **revision activation fails categorically**. In our pilots this single pathway failure explained three seemingly unrelated phenomena: disruption-triggered revision, job-arrival revision (starving all late arrivals: throughput $24.4 \rightarrow 0.24$ jobs/ksteps), and periodic refresh. Simultaneously, one-shot plans looked deceptively healthy—slack absorbs outages (4–14% makespan growth under a 200-step outage) and duration variance ($D \in [1.00, 1.15]$)—so nothing forced anyone down the revision path. We consider this a cautionary tale for evaluating closed-loop machinery *only* under weak disruptions.

5 Soft Commitments as Conservative Projection

5.1 Commitment projection

The repair is best understood not as a patch but as an abstraction. An *execution commitment* for a frozen operation p is the triple $\kappa(p) = (\mu, l, r)$: the committed resource μ (the machine where p will run, hence where its successor’s input material will land), the material source l (that machine’s cell), and the earliest-safe-release r (the earliest step at which the successor of p may begin). A *commitment projection* Π maps the execution state to the residual problem instance by turning each commitment into (i) a release bound $\bar{r} \geq r$ for the successor,

(ii) a source $\bar{l}$ for its transfer request, and (iii) machine-availability offsets.

Two projections are of interest. An *exact* projection achieves $\bar{r} = r$ and $\bar{l} = l$; a *conservative* projection requires only $\bar{r} \geq r$ with $\bar{l} = l$ exact. The legacy encoder is an *all-or-nothing* projection: any commitment whose (μ, l) it cannot derive exactly is excluded from the encoding and vetoes activation outright. Soft commitments complete the projection:

Exact (μ, l) resolution. Operation objects lack an assigned machine before processing starts, but the information exists elsewhere: a `MACHINE_QUEUE` operation sits in exactly one machine’s input queue; a `TRANSPORT` operation’s delivery destination is on its transport record. Matching destinations to machine cells resolves (μ, l) exactly, so only r need be bounded.

Bounded releases. For predecessor p committed to machine $m = \mu(p)$, the successor’s release bound is

$$p \in \text{MACHINE_QUEUE} : \bar{r}(o) \geq \text{avail}(m) + p_m, \quad (1)$$

$$p \in \text{TRANSPORT} : \bar{r}(o) \geq \Delta_{\text{pen}} + p_m, \quad (2)$$

where p_m is p ’s duration on m (max over alternatives if unassigned), $\text{avail}(m)$ accounts for repair time and the current operation, and Δ_{pen} is the single *penalty parameter*—a generous travel allowance (default 200 steps for maps up to 32×32). Bound (1) is exact when p is next in its FIFO queue and a relaxation otherwise (work ahead of p is not yet included; §5.2 shows why the slack is safe); bound (2) is conservative under Assumption A1 below. Material sources use the exactly resolved machine; client-only configuration keys are stripped from the service payload.

5.2 Soundness

We separate two guarantees. Let the *stack release semantics* be: **S1** release times gate the dispatch of transport requests only; **S2** an AGV’s pickup physically blocks at the source until the material exists; **S3** a machine starts an operation only when it has entered its input queue, which happens only on delivery.

Proposition 1 (Execution soundness, unconditional). *Under S1–S3, every conservative projection preserves material precedence at execution: no operation starts before its predecessor’s material is delivered, for any $\bar{r} \geq 0$ (in particular, for any Δ_{pen}).*

Proof sketch. By S1 an under-estimated $\bar{r}$ can only dispatch a transfer early; by S2 that transfer waits at its source until the predecessor completes; by S3 processing cannot begin before delivery. The failure mode of a too-small bound is an idle AGV camping at a source (congestion, under-utilization), never a premature start. $\square$

Proposition 1 is what makes the penalty knob safe to sweep: mis-setting Δ_{pen} degrades performance, never correctness. The second guarantee is about the plan, not the execution:

Proposition 2 (Plan-time validity, conditional). *Assume A1 (remaining transport time of any committed transfer $\leq \Delta_{\text{pen}}$) and FIFO input queues. Then $\bar{r}(o) \geq r(o)$ for all successors, so every start time in the revised plan is a feasible lower bound: the plan executes without release-induced stalling beyond the bounds’ slack.*

Under A1 violation (extreme congestion), behavior falls back to Proposition 1: the plan goes stale but execution stays safe—a two-level guarantee we exploit when sweeping Δ_{pen} down to 10 steps (§7).

Implementation. The abstraction instantiates in ~ 90 lines of client-side encoder changes in our stack, opt-in via a solver-config flag, leaving legacy behavior bit-identical. The abstraction itself is stack-independent: any dispatch-by-release simulator with physically enforced material precedence (S1–S3) admits the same construction.

6 Recovery Orchestrator

On top of unblocked revisions we instantiate the policy space $\mathcal{O}(\tau, \kappa)$: trigger $\tau \in \{\text{never}, \text{event}, \text{periodic-}K, \text{myopic}\}$ and scope $\kappa \in \{\text{full}, \text{partial}\}$ (scoped repair around a failed machine), with an escalation ladder (full $\rightarrow$ partial $\rightarrow$ defer) and a minimum replan interval. Policies serving as treatments: GREEDY-REACTIVE (rule-based, replans by construction), STATIC (one-shot CP-SAT, $\tau=\text{never}$), FULL, PARTIAL, and MYOPIC—a value-aware trigger that, at each qualifying event, exploits the engine’s deterministic snapshot/replay to roll out both branches for H steps under identical exogenous events and replans only if $\hat{V}(s) = \text{prog}_B - \text{prog}_A > \lambda$, where progress is completed operations (ties broken by accumulated processing time). This is the simplest instantiation of the when-to-replan decision policy our results call for; it requires no learning. One caveat: replay restores the world RNG, so under stochastic scenarios (S_3) the rollouts see the actual future tape—a mild clairvoyance in that arm; under deterministic scenarios (S_0, S_1, S_4) the rollouts are leak-free by construction, and E7 headline comparisons hold within both groups. A reseeded leak-free variant (independent rollout tape, common random numbers across branches) is left as future work.

7 Experiments

Benchmark. Brandimarte instances $\times$ map families (corridor maze / open random) $\times$ fleet sizes, on a full-

stack coupled factory simulator: a discrete-event grid factory with a CP-SAT scheduling service and a rolling EECBS routing service. Episodes are subprocess-isolated with hard timeouts; all runs, seeds, and manifests are recorded. The baseline commitment encoding (§4) is the system’s standard mode; soft commitments are an opt-in flag. **Campaign (1209 episodes total):** a 948-episode main factorial campaign (E1–E4) plus 261 episodes of route-backend sensitivity, arrival factorials, and value-aware-trigger comparison (E5–E7). *E1* main factorial: $\{\text{mk01}, \text{mk02}, \text{mk05}, \text{mk07}\} \times \text{maze} \times K \in \{2, 4, 6\} \times \{S_0, S_1=\text{m_down@150}, S_4=\text{m_down@80} \times 200, S_3=\text{stochastic}\} \times 4 \text{ policies} \times 3 \text{ seeds}$. *E2* repeats a subset on the open map. *E3* legacy-commitment control (trigger-level activation success). *E4* penalty sweep $\Delta_{\text{pen}} \in \{50, 100, 200, 400\}$, plus a $\Delta_{\text{pen}}=10$ stress test backing Proposition 1. *E5* route-backend sensitivity (EECBS vs Python LaCAM/PIBT). *E6* arrivals factorial (legacy vs soft commitments; arrival-revision failures). *E7* myopic value-aware trigger vs static/full. We report *trigger-level activation success* (share of triggered revisions that activate; E1–E3) and *arrival-revision failures* (failed arrival revisions; E6) as distinct metrics.

7.1 Results

R1: Soft commitments fix the mechanism (E3 vs E1/E2). Under legacy encoding, revision activation succeeds only $56/(56+50) = 53\%$ of trigger opportunities; with soft commitments, $1125/(1125+17) = \mathbf{99\%}$ ($216+216$ episodes, 2401 triggered revisions in total). The mechanism bottleneck is real and the fix removes it.

R2: With the mechanism fixed, passive robustness dominates disruption response (E1+E2, 864 episodes). Table 1 reports makespans with right-censored episodes (timeouts or non-completion) imputed at the 4096-step cap, which removes survivorship bias. STATIC beats FULL/PARTIAL *in every disruption scenario*: paired win rate 143:12 (61 ties, $n=216$), Wilcoxon $p < 10^{-4}$ on each of S_1, S_4, S_3 . Disruption regret (Table 2) tells the same story: one-shot plans finish within $1.0\text{--}1.06\times$ of their no-disruption makespan even under 200-step outages, whereas event-triggered revision with our conservative bounds pays $1.25\text{--}2.25\times$. Slack absorbs bounded disturbances; replanning under them only re-imposes conservatism.

R3: Scope equivalence. FULL and PARTIAL produced *identical* episodes on every one of 216 paired runs: under single-machine failures the affected-successor scope already spans the entire reschedulable set, so scoped repair and full revision coincide. Scope differentiation requires concurrent multi-machine failures—outside our scenario set.

Scen	greedy	static	full	partial
S_0	2136	1117	1117	1117
S_1	2115	1003	1309	1309
S_4	2201	1017	1333	1333
S_3	2464	1618	2281	2281

Table 1: Makespan (mean $\pm$ std collapsed to mean; censored episodes imputed at 4096). STATIC dominates all disruption scenarios.

Scen	static	full/partial	n
S_1 (outage 40)	0.99 $\pm$ 0.03	1.25 $\pm$ 0.14	46–48
S_4 (outage 200)	1.06 $\pm$ 0.11	1.26 $\pm$ 0.12	44–48
S_3 (stochastic)	1.03 $\pm$ 0.05	2.25 $\pm$ 1.16	37–39

Table 2: Disruption regret $C_{\max}(S)/C_{\max}(S_0)$ (finished episodes).

R4: The penalty knob is real but locally flat (E4). Sweeping $\Delta_{\text{pen}} \in \{50, 100, 200, 400\}$ on $\{\text{mk01}, \text{mk05}\} \times \{S_1, S_4\}$ shows no monotone trend (best mean at 200: 899/817 vs 939–1067 elsewhere; $n=4$ –6 per cell after censoring). A stress run at $\Delta_{\text{pen}}=10$ —twenty times below the default and far under any travel bound—completes all episodes with revisions activating normally and no execution violations, degrading only in makespan (1392–3801 vs $\sim$ 1400 at the default), exactly as Proposition 1 predicts: early-dispatched transfers wait at their sources rather than start prematurely. The knob exists and its response is measurable; adaptive or per-commitment penalties remain open.

R7: A value-aware trigger improves on both extremes (E7, 108 episodes). Under local A* routing (the regime where the routing layer is weakest and replanning has the most room to matter), we compare STATIC, FULL, and MYOPIC on an identical factorial $\{\text{mk01}, \text{mk05}\} \times K \in \{4, 6\} \times \{S_1, S_4, S_3\} \times 3$ seeds. **myopic wins overall:** censored-imputed mean 2876 vs 3018 (STATIC) and 3466 (FULL); censoring 14 vs 15/20; paired 14:9 (13 ties) over static and 21:6 (9 ties) over full. The mechanism is surgical: across 453 trigger evaluations it replanned only 6 times (1.3%)—almost always endorsing the law of R2 (leave the plan alone), yet capturing the rare states where revision pays (largest gain under S_4 : 2039 vs 2296). A no-learning, snapshot-based lookahead thus instantiates the when-to-replan decision policy our diagnosis calls for; in this regime it improves on both never-replan and always-replan with paired significance, though the margin is modest against static (14:9) and the comparison is limited to local A* routing.

R5: Arrivals (E6, 81 controlled episodes). The regime where slack cannot help is work absent from the plan. On the pre-fix stack we observed total ar-

rival starvation (throughput $24.4 \rightarrow 0.24$ jobs/ksteps); the controlled E6 factorial ($\{\text{mk01}, \text{mk02}, \text{mk05}\} \times \text{cadence} \{10..100\} \times \{\text{legacy}, \text{soft}\} \times 3$ seeds, plus batch anchors) isolates what the *commitment encoding* itself contributes on the repaired engine: legacy revisions still fail 1–18 times per cell whenever an arrival lands mid-commitment, whereas soft commitments fail **never** (0 across all 72 streaming episodes). Operationally: soft guarantees completion everywhere (10/10, 15/15 jobs; the worst legacy cell drops to 7/10) and lifts throughput on the largest shop by 23–31% (mk05: $8.1 \rightarrow 10.5$ at cadence 20, $9.6 \rightarrow 12.5$ at 50), while smaller shops are at parity—arrivals benefit from unblocked reliability rather than from dramatic rescue.

R6: Route-backend sensitivity (E5, 72 episodes). Swapping only the routing backend behind the identical protocol dramatically reshapes the coupled stack: with the production C++ EECBS service, STATIC completes these cells at $\approx$ 400–800 steps; with the reference Python LaCAM server every metric degrades to near-livelock ($\approx$ 3900, 4/36 errors), and the Python PIBT server fails outright (22/36 errors even at a relaxed 3s planning budget). The rolling closed loop is bottlenecked by *per-replan latency* of the routing backend—consistent with the routing layer being the binding constraint of hierarchical decomposition, and a caution for evaluating learned MAPF backends inside coupled simulators.

8 Discussion: When *Not* to Replan

Our factorial evidence inverts the usual closed-loop pitch. Once the commitment encoding is fixed and revisions actually execute, the *correct* decision in every disruption scenario we could construct is to leave the one-shot plan alone: slack plus release-by-plan semantics absorbs 200-step outages within 6%, and conservative revision consistently pays a 25–125% premium for responsiveness that nothing needs. The regime where replanning is structurally indispensable is *arrivals*—work absent from the plan that no amount of slack can absorb. This reframes closed-loop factory scheduling: the research question is not a better trigger policy but a *decision policy* for when revision is worth its conservatism, plus commitment encodings whose penalty adapts (per-commitment travel bounds, learned penalties). Our unblocked stack makes that question measurable for the first time.

9 Conclusion

In coupled scheduling-and-path-finding stacks, the obstacle to closed-loop operation is not the replanning policy but the commitment encoding; a sound conservative

relaxation lifts activation from 53% to 99% and eliminates revision failures outright. With the pathway repaired, a 1209-episode factorial study shows that disruption response should, counterintuitively, stay open-loop: slack dominates conservatism. Replanning earns its keep through reliability under arrivals—guaranteed completion and up to 31% throughput on larger shops—and a no-learning, lookahead value-aware trigger captures the best of both regimes surgically (1.3% replan rate, improving on both extremes in the studied regime). The remaining open problem is adaptive commitment penalties and richer value estimates, which our unblocked stack now makes measurable.

References

- [1] Anton Andreychuk, Konstantin Yakovlev, Dor Atzmon, and Roni Stern. MAPF-GPT: Imitation learning for multi-agent pathfinding at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.
- [2] Paolo Brandimarte. Routing and scheduling in a flexible job shop by tabu search. *Annals of Operations Research*, 41(3):157–183, 1993.
- [3] Stéphane Dauzère-Pérès and Jan Paulli. The flexible job shop scheduling problem: a review. *European Journal of Operational Research*, 2024.
- [4] Carmel Domshlak, Izai Hatzzy, and Alexander Shleyfman. Satisficing planning with commitment. In *ICAPS Workshop on Heuristics and Search for Domain-Independent Planning*, 2013.
- [5] Maria Fox, Alfonso Gerevini, Derek Long, and Ivan Serina. Plan stability: Replanning versus plan repair. *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2006.
- [6] Jiaoyang Li, Zhe Chen, Daniel Harabor, Peter J. Stuckey, and Sven Koenig. MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.
- [7] Jiaoyang Li, Daniel Harabor, Peter J. Stuckey, and Sven Koenig. EECBS: A bounded-suboptimal search for multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [8] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W. Durham, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding in large-scale warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [9] Rui Li et al. A new framework for job shop integrated scheduling and vehicle path planning problem. In *Sensors*, volume 26, 2026.
- [10] Lei Lu et al. Integrated optimization of dynamic flexible job shop scheduling considering AGV breakdowns. *Computers & Industrial Engineering*, 2025.
- [11] Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2017.
- [12] Keisuke Okumura. LaCAM: Search-based algorithm for quick multi-agent pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [13] Tilman Peltzer, Ruben vander Wedden, Elrike Adelbrecht, Bo Bonet, Ioannis Rexakis, and Michail G. Lagoudakis. STT-CBS: A conflict-based search algorithm for multi-agent path finding with stochastic travel times. In *arXiv:2004.08025*, 2020.
- [14] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. In *Artificial Intelligence*, volume 219, pages 100–132, 2015.
- [15] Alexey Skrynnik, Anton Andreychuk, Maria Nesterova, Konstantin Yakovlev, and Aleksandr Panov. Decentralized MCTS: The case of the partially observable multi-agent pathfinding problem. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [16] Igghard Smit et al. Graph neural networks for job shop scheduling problems: a review. *Computers & Operations Research*, 2025.
- [17] Bin Xin et al. A review of flexible job shop scheduling problems considering processing machines and transportation vehicles. *Frontiers of Information Technology & Electronic Engineering*, 2025.
- [18] Cong Zhang, Wen Song, Zhiguang Cao, Jie Zhang, Puay Siew Tan, and Chi Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. In *Advances in Neural Information Processing Systems*, 2020.